FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress 89%

Go Back

Back To Practice

Q1

Save

Completed

First Fit Contiguous Memory Allocation

Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.

Input Format

1. The first line contains an integer m representing the number of memory blocks.
2. The next line contains m integers representing the sizes of each memory block.
3. The next line contains an integer n representing the number of processes.
4. The next line contains n integers representing the sizes of each process.

Output Format

Select Language : C (GCC 9.2.0)

```c
#include <stdio.h>
int main() {
    int m, n;
    printf("Enter number of memory blocks:\n");
    scanf("%d", &m);
    int blockSize[m];
    int originalBlockSize[m];
    printf("Enter size of each block:\n");
    for(int i = 0; i < m; i++) {
        scanf("%d", &blockSize[i]);
        originalBlockSize[i] = blockSize[i];
    }
    printf("Enter number of processes:\n");
    scanf("%d", &n);
    int processSize[n];
    printf("Enter size of each process:\n");
    for(int i = 0; i < n; i++) {
        scanf("%d", &processSize[i]);
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

Custom Test

Success

Input :

Expected Output :

Completed

First Fit Contiguous Memory Allocation

Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.

Input Format

1. The first line contains an integer m representing the number of memory blocks.
2. The next line contains m integers representing the sizes of each memory block.
3. The next line contains an integer n representing the number of processes.
4. The next line contains n integers representing the sizes of each process.

Output Format

- For each process, the program prints whether the process is

Select Language : C (GCC 9.2.0)

```
    for(int i = 0; i < n; i++) {
        int allocated = 0;
        for(int j = 0; j < m; j++) {
            if(blockSize[j] >= processSize[i]) {
                int fragment = blockSize[j] - processSize[i];
                printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
                       i + 1, processSize[i], j + 1,
                       originalBlockSize[j], fragment);
                blockSize[j] -= processSize[i];
                allocated = 1;
                break;
            }
        }
        if(!allocated) {
            printf("Process %d of size %d is not allocated\n",
                   i + 1, processSize[i]);
        }
    }
    return 0;
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

Custom Test

Success

Input :

Expected Output :